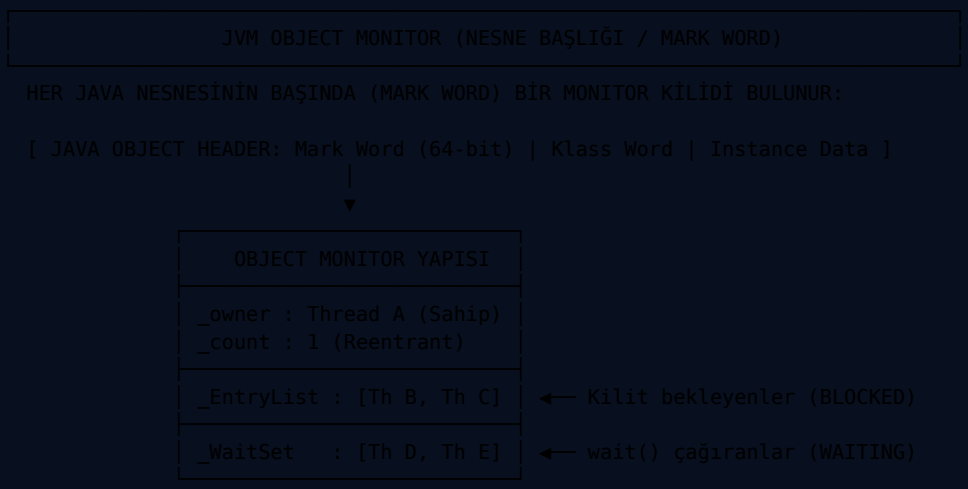
13

BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

synchronized Anahtar Kelimesi: Intrinsic Lock & JVM Object Monitor Mimarisi

The synchronized Keyword: Intrinsic Locks & JVM Object Monitors

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Her Java nesnesi, kapısında özel bir anahtar deliği olan bir oda gibidir. Bir odaya `synchronized` ile girdiğinizde (monitorenter), içeriden kapıyı kilitlersiniz. Odanın sahibi siz olursunuz (_owner). Kapıda iki tane bekleme salonu vardır: 1) Koridor (_EntryList): İçeri girmek için sıraya girenler (BLOCKED). 2) Balkon (_WaitSet): İçerideyken 'Patron haber verene kadar burada dinleniyorum' diyerek kenara çekilenler (WAITING). Siz odadan çıkarken (monitorexit) anahtarı bırakırsınız ve koridordaki bir kişiyi içeri alırsınız!

🇬🇧 ENGLISH (ELI5)

Every Java object is born with a private hotel room keyhole inside its memory header (Mark Word). When a thread enters a `synchronized` block, it executes `monitorenter`, turning the deadbolt from the inside and claiming ownership (`_owner`). Outside sit two distinct waiting rooms: 1) The Hallway (`_EntryList`): Threads aggressively competing to enter the room (BLOCKED). 2) The Balcony (`_WaitSet`): Threads that were inside, voluntarily surrendered the key via `wait()`, and are resting until signaled (WAITING). Exiting triggers `monitorexit`, unlocking the door for hallway contenders.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Object Monitor ve Bytecode Derinliği: 1) Mark Word: 64-bit HotSpot JVM'de nesne başlığının en alt 2-biti kilit durumunu tutar (01: Kilitsiz, 00: Hafif Kilit, 10: Ağır Monitör Kilidi). 2) Reentrancy (Yeniden Girilebilirlik): Bir thread zaten sahip olduğu bir kilide tekrar girerse (örneğin synchronized metodun içinden başka bir synchronized metot çağırmak), JVM `_count` sayacını 1 artırır; kilitlenip deadlock olmaz. Çıkarken de sayaç düşer. 3) Derleyici Garantisi: `synchronized` bloğu javac tarafından try-finally bloğu gibi derlenir. Metot içinde beklenmedik bir RuntimeException veya Error fırlasa dahi JVM `monitorexit` talimatını kesinlikle çalıştırır; kilit asla askıda kalmaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Monitor Internals & Bytecode: 1) Mark Word State Bits: On 64-bit architectures, the lowest two bits designate monitor states (01 = Unlocked, 00 = Light-weight/Thin, 10 = Heavyweight Inflated Monitor). 2) Reentrancy Mechanism: If Thread A holds a monitor and enters another synchronized block on the same object, the JVM simply increments `_count`. It does not re-acquire or block, preventing self-deadlock. The monitor is released when `_count` hits zero. 3) Bytecode Invariant: javac generates dual `monitorexit` opcodes: one for normal control flow completion and another wrapped in an all-encompassing exception handler, ensuring monitors are guaranteed released even under fatal panics.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Synchronized Reentrancy ve Monitor Davranışı
public class SynchronizedMonitorDemo {
    private final Object lock = new Object();
    private int sharedBalance = 1000;

    public void withdraw(int amount) {
        synchronized (lock) { // monitorenter
            IO.println("Kilit alındı: " + Thread.currentThread().getName());
            auditLog(); // Reentrant çağrı (Aynı kilit, sayaç 2 olur!)
            sharedBalance -= amount;
        } // monitorexit
    }

    private void auditLog() {
        synchronized (lock) { // Aynı thread olduğu için bloke olmaz!
            IO.println("Audit kaydı işlendi. Kalan bakiye: " + sharedBalance);
        }
    }

    void main() {
        var demo = new SynchronizedMonitorDemo();
        demo.withdraw(250);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'synchronized(new Object()) veya synchronized(new String("lock")) yazarsak ne olur?' Yanıt: 'Her thread farklı bir nesne referansı üzerinde kilitleneceği için hiçbir senkronizasyon sağlanamaz! Kilit nesneleri DAİMA `private final Object lock = new Object();` olarak tanımlanmalıdır.'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Junior Trap in Interviews: 'What is the catastrophe of `synchronized(new Object())` or `synchronized("literal")`?' Answer: '`new Object()` creates a brand-new monitor per invocation, providing ZERO mutual exclusion. Locking on interned Strings risks accidental cross-library deadlocks. Always declare `private final Object lock = new Object();`.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Her Java nesnesi başlığında (Mark Word) bir Object Monitor taşır.
- synchronized kilitleri reentrant'tır; aynı thread aynı kilide defalarca girebilir.
- Kilit nesneleri referans değişimini önlemek için daima private final olmalıdır.

🎯 Key Takeaways for the Interview

- Every Java object carries an intrinsic monitor inside its Mark Word header.
- Intrinsic locks are fully reentrant, incrementing an internal recursion counter.
- Lock target instances must always be private and final to prevent reference mutation.